Project 2: Explainability — Presentation Notes

1. What the App Does

This app trains two types of machine learning models and then explains their predictions in human-understandable ways.

Instead of just asking *"what does the model predict?"*, the app answers *"why does the model predict that?"* and *"what would need to change to get a different prediction?"*

It has four sections:

- **Home** — dataset overview and key concept definitions
- **Train** — choose a model and use a slider to balance accuracy vs. simplicity
- **Counterfactuals** — see what small changes would flip a prediction
- **PDP / ALE** — see how individual features influence predictions on average

2. What Dataset is Used

The app uses the **Palmer Penguins** dataset.

It contains measurements of **333 penguins** from three islands in Antarctica, collected between 2007 and 2009. After removing rows with missing values, each row represents one penguin with 7 features:

Feature	Type	Example
bill_length_mm	Numerical	39.1 mm
bill_depth_mm	Numerical	18.7 mm
flipper_length_mm	Numerical	181 mm
body_mass_g	Numerical	3750 g
island	Categorical	Torgersen
sex	Categorical	male
year	Categorical	2007

3. What the Target Is

The target column is **species** — the model predicts which of three penguin species a given penguin belongs to:

- **Adelie**
- **Chinstrap**
- **Gentoo**

This is a multi-class classification problem.

4. What Decision Tree Explains

A Decision Tree makes predictions by following a series of yes/no questions about the features — like a flowchart.

> *"Is the flipper length greater than 206mm? → If yes → probably Gentoo."*

Why it is explainable: The decision path is transparent. You can trace exactly which questions were asked and which thresholds were crossed to reach a prediction.

Technical note: Complexity = **number of leaves** (end nodes of the tree). More leaves = more decision paths = more complex model.

5. What Logistic Regression Explains

Logistic Regression assigns a **weight (coefficient)** to each feature. A larger weight means that feature has a stronger influence on the prediction. A zero weight means the model ignores that feature entirely.

> *"A longer flipper increases the probability of Gentoo by coefficient X."*

Why it is explainable: The coefficients show directly how much each feature matters and in which direction.

Technical note: Complexity = **number of non-zero coefficients**. The model uses L1 regularisation (via the `saga` solver with `l1_ratio=1.0`), which drives unimportant feature weights to exactly zero — this is called a sparse model.

---

6. What Lambda Controls

λ (lambda) is a slider from 0 to 1 that controls the trade-off between **accuracy** and **simplicity**.

λ value	Effect
λ = 0	Only accuracy matters — pick the most accurate model, no matter how complex
λ = 0.5	Balance accuracy and simplicity equally
λ = 1	Only simplicity matters — pick the simplest model, even if less accurate

This reflects a key idea in Human-Centric AI: **a simpler model is easier for humans to understand and trust**, even if it is slightly less accurate.

---

7. How the Selected Model is Chosen

For each model type, the app trains **8 candidate models** with different hyperparameters:

- Decision Tree: 8 different maximum depths (1, 2, 3, 4, 5, 7, 10, unlimited)
- Logistic Regression: 8 different regularisation strengths (C = 0.01 → 30)

Each candidate is evaluated using this objective function:

```
objective = (1 − accuracy) + λ × normalised_complexity
```

Lower objective = better. The candidate with the lowest objective is selected.

Normalised complexity scales all complexity values to the range [0, 1] so that λ has the same meaning regardless of model type.

---

8. What Counterfactual Explanations Mean

A counterfactual answers the question:

> *"What is the smallest realistic change to this penguin's features that would make the model predict a different species?"*

Example: A penguin is predicted as Adelie. The counterfactual might show: *"If the flipper length were 208mm instead of 181mm, and the body mass were 5200g instead of 3750g, the model would predict Gentoo."*

How it works in this app:

1. Sample 5000 random neighbours around the original penguin using Gaussian noise (scaled by the feature's Median Absolute Deviation)
2. Categorical features (island, sex, year) are randomly resampled from real dataset values — never invented
3. Filter: keep only neighbours predicted as the desired target class
4. Rank by MAD-weighted L1 distance — closer = better counterfactual
5. Return the top 5

The yellow cells in the table show which features changed.

---

9. What PDP Shows

PDP = Partial Dependence Plot

PDP shows how the model's **average predicted probability** changes as one feature varies across its full range, while all other features are held at their real observed values.

> *"On average, what happens to the probability of each species as flipper_length_mm increases from 170mm to 230mm?"*

Implemented manually: For each grid value, the app copies the full dataset, forces the selected feature to that value for all rows, calls `predict_proba`, and averages the results. No external PDP library is used.

Limitation: PDP can be misleading when features are correlated with each other, because it may ask the model to predict on combinations of feature values that never appear in real data.

10. What ALE Shows

ALE = Accumulated Local Effects

ALE solves the main limitation of PDP. Instead of forcing a feature to a value across the entire dataset, ALE only looks at **small local changes** within bins where the data actually exists.

> *"Within the group of penguins with flipper length around 190mm, what happens to the prediction when we move the feature slightly up or down?"*

Implemented manually using bin-based ALE:

1. Divide the feature into quantile bins
2. For each bin, find the real data points inside it
3. Create two copies: one with the feature set to the lower bin edge, one to the upper edge
4. Compute `predict_proba` for both copies and take the difference
5. Accumulate the differences across bins (running sum)
6. Centre the result around zero

Technical note: Logistic Regression is mathematically differentiable, so its ALE could in theory be computed using exact partial derivatives. However, Decision Trees are piecewise constant and not differentiable. This app uses **bin-based ALE for both models** to keep the interface model-agnostic — the same method works the same way regardless of which model is selected.

---

11. Why This is Human-Centric AI

Human-Centric AI is about designing AI systems that are transparent, understandable, and trustworthy for the humans who use them.

This app demonstrates several HCAI principles:

Principle	How this app addresses it
Transparency	PDP and ALE are computed manually — the algorithm is visible in the code, not hidden in a library
Understandability	Counterfactuals answer a natural human question: "What would need to change?"
User control	The λ slider lets the user decide how much they value accuracy vs. simplicity
Beginner-friendliness	Every section includes plain-language explanations alongside the technical results
Model comparison	Both Decision Tree and Logistic Regression are offered, so the user can choose the model type they understand best
Graceful failure	If no counterfactuals are found, the app gives friendly suggestions — no raw errors

The core philosophy: **a model that a human can understand and question is more valuable than a slightly more accurate black box.**